Slack Sentiment Analysis System

Complete End-to-End Implementation Plan

LLM-Only | Node.js-Only | Supabase Architecture

Version 3.0 — Supabase Revision

March 2026

Table of Contents

1. Problem Statement
2. System Overview
3. Requirements Analysis
4. System Boundaries
5. High-Level Architecture
6. Data Flow — End-to-End
7. Supabase Platform Architecture
8. Database Schema Design
9. Event Ingestion Pipeline
10. Backfill Engine
11. Thread Reconstruction
12. Text Normalization Pipeline
13. LLM Inference Design
14. Conditional LLM Gating
15. Context Management and Memory
16. Incremental Summarization
17. User Profile Enrichment
18. Alerting Service
19. Analytics REST API
20. Queue Architecture (pg-boss over Supabase)
21. Concurrency Model
22. Security Architecture
23. Edge Cases and Failure Modes
24. Worst-Case Scenario Analysis
25. Cost Model and Budget Control
26. Observability and Monitoring
27. Deployment Strategy
28. Phased Implementation Roadmap
29. Technology Stack Summary
30. Appendix: Complete Type Definitions

---

1. Problem Statement

1.1 The Core Gap

Organizations using Slack as their primary communication channel with clients lack real-time visibility into the emotional tone and sentiment of incoming messages. Customer-facing teams miss signals of frustration, dissatisfaction, or escalation embedded in natural language, emoji usage, and conversational context. This results in delayed responses to at-risk accounts, missed intervention opportunities, and inability to quantify client satisfaction trends.

1.2 What This System Solves

This system is not a chatbot. It is a **passive, multi-channel, context-aware conversational intelligence layer** that sits over Slack channels and continuously:

- Intercepts messages in real time via Slack Events API.
- Reconstructs full conversation history when joining a channel mid-stream.
- Builds and maintains per-channel isolated memory (thread structure, running summaries, participant maps, decision logs).
- Performs emotion and sentiment analysis using a hosted LLM (OpenAI or Anthropic) via API.
- Stores structured analytical results for dashboards, alerting, and trend monitoring.
- Triggers alerts when negative sentiment exceeds configurable thresholds.
- Tracks LLM usage costs per workspace and channel.

1.3 Why LLM-Only, Node.js-Only, Supabase

LLM-Only: No local transformer models, no Python services, no GPU requirements. The LLM is accessed as an external API. This eliminates model hosting, cold-start latency, and the Python ML ecosystem from the operational surface. An LLM provides superior contextual reasoning — it processes entire threads, understands sarcasm, detects implicit frustration, and generates explanations.

Node.js-Only: A single-language stack eliminates cross-service HTTP calls, reduces Docker image count, simplifies CI/CD, and removes an entire class of deployment and debugging complexity. All text preprocessing (markup stripping, emoji conversion, normalization) is achievable in Node.js.

Supabase: Replaces self-managed PostgreSQL with a managed platform that provides:

- Hosted PostgreSQL 15 with pgvector extension pre-installed.
- Built-in connection pooling via Supavisor (replaces PgBouncer).
- Row Level Security (RLS) for multi-tenant data isolation.
- pg_cron for scheduled jobs (data retention, cost aggregation).
- Supabase client SDK (`@supabase/supabase-js`) for simplified queries.
- Direct PostgreSQL connection string for pg-boss queue operation.
- Dashboard for database management and monitoring.
- Edge Functions for potential future serverless workers.
- Eliminates database provisioning, backup management, and scaling operations.

Trade-offs explicitly accepted:

- Higher per-request LLM cost vs infrastructure cost (mitigated by conditional gating).
- LLM latency of 1-3 seconds per call (acceptable because processing is asynchronous).
- LLM confidence values are self-estimated, not calibrated (acceptable for business use).
- Supabase introduces a managed dependency (mitigated by using standard PostgreSQL — migration to self-hosted Postgres requires only changing the connection string).

---

2. System Overview

2.1 One-Line Summary

A single Node.js bot instance, invited into multiple Slack channels, that maintains per-channel isolated context memory with persistent Supabase storage, performs conditional LLM-based emotion analysis, and exposes analytics via REST API.

2.2 Core Behavioral Contract

When the bot is invited to a channel:

1. It detects the `member_joined_channel` event.
2. It transitions the channel from `UNINITIALIZED` to `INITIALIZING`.
3. It backfills up to 30 days of message history from the Slack Web API.
4. It reconstructs all thread structures (parent messages + replies).
5. It resolves user display names for all participants.
6. It generates an initial running summary via hierarchical LLM compression.
7. It marks the channel as `READY`.
8. From that point forward, every new message is captured in real time.

For each new message after initialization:

1. The message is stored in Supabase immediately.
2. A lightweight risk heuristic evaluates the message text.
3. If the risk score exceeds the threshold, or if enough messages/time have elapsed since the last LLM call, the system triggers an LLM analysis.
4. The LLM receives a structured context pack (running summary + recent messages + relevant historical context).
5. The LLM returns structured JSON with dominant emotion, confidence, escalation risk, and explanation.
6. Results are stored, channel state is updated, and alerts fire if thresholds are exceeded.

The bot never posts messages. It is purely analytical.

---

3. Requirements Analysis

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	Receive Slack message events via Events API webhook endpoint	P0
FR-02	Verify Slack request signatures using HMAC-SHA256	P0
FR-03	Perform idempotency checks using Slack event_id to reject duplicates	P0

FR-04	Detect bot join events and trigger channel initialization	P0
FR-05	Backfill historical messages via conversations.history with cursor pagination	P0
FR-06	Fetch all thread replies via conversations.replies for threads with reply_count > 0	P0
FR-07	Reconstruct thread graph (parent-child relationships)	P0
FR-08	Store raw messages in Supabase PostgreSQL	P0
FR-09	Resolve user display names via users.info API with caching	P1
FR-10	Normalize Slack-specific text formatting (strip markup, convert emoji, preserve sentiment signals)	P0
FR-11	Call hosted LLM (OpenAI or Anthropic) with structured prompt enforcing JSON output	P0
FR-12	Validate LLM JSON response against schema using ajv	P0
FR-13	Store structured analysis results (emotion, confidence, escalation risk, explanation)	P0
FR-14	Implement conditional LLM gating (not per-message)	P0
FR-15	Maintain per-channel running summary via incremental summarization	P1
FR-16	Store context documents with vector embeddings for semantic retrieval	P1
FR-17	Trigger threshold-based alerts on high-risk sentiment	P1
FR-18	Track LLM token usage and estimated cost per request	P1
FR-19	Expose REST API for channel state, messages, sentiment results, trends, costs	P0
FR-20	Support manual LLM analysis trigger via API endpoint	P1
FR-21	Buffer real-time messages during backfill; process after backfill completes	P0
FR-22	Support multi-workspace via workspace_id partitioning	P2

3.2 Non-Functional Requirements

ID	Requirement	Target
NFR-01	Webhook must return 200 OK within 3 seconds (Slack timeout)	< 3s
NFR-02	Support sustained ingestion of 500+ messages/minute per workspace	500 msg/min
NFR-03	Guarantee exactly-once processing despite Slack's at-least-once delivery	Zero duplicates
NFR-04	Graceful degradation: if LLM is unavailable, messages queue and retry	No data loss
NFR-05	Signature verification on every request	100%
NFR-06	No sensitive message content in application logs	Zero leakage
NFR-07	LLM provider swappable without modifying ingestion or storage code	Env var change
NFR-08	P95 end-to-end analytics availability under 10 seconds after message arrival	< 10s
NFR-09	Channel initialization (30-day backfill) completes within 2-10 minutes	< 10 min
NFR-10	All state persists across application restarts	Zero state loss
NFR-11	Per-channel context isolation — no cross-channel data leakage	Zero leakage
NFR-12	Raw message retention: 90 days. Summaries retained long-term.	90 days + forever

3.3 Assumptions

Item	Assumption
Message volume	100-500 messages/minute per workspace during peak
LLM cost budget	~$0.01-$0.03 per analysis (GPT-4o-mini or Claude Haiku)
Multi-workspace	Supported via workspace_id but single workspace for MVP
Message types	Only human-sent text messages analyzed. Bot messages, file uploads, reactions excluded
Language	English-only at MVP. LLMs support multilingual as future extension

Thread depth	Thread context limited to last 20 messages per LLM call
Dashboard	Out of scope. Analytics exposed via API endpoints
Bot behavior	Passive/analytical only. No auto-posting in Slack
Supabase plan	Pro plan or above for pgvector, pg_cron, and sufficient connection pooling

---

4. System Boundaries

4.1 In Scope

- Slack Events API integration, signature verification, idempotent event ingestion.
- Channel initialization with historical backfill on bot join.
- Thread structure reconstruction (conversations.history + conversations.replies).
- User profile resolution and caching.
- Asynchronous message processing pipeline with pg-boss queue over Supabase.
- Text normalization in Node.js (Slack markup stripping, emoji conversion).
- LLM-based emotion classification via hosted API (OpenAI primary, Anthropic alternative).
- Conditional LLM gating (threshold-based + risk heuristic + manual trigger).
- Per-channel isolated context management (running summary, key decisions, sentiment timeline).
- Incremental summarization with vector embeddings stored in Supabase pgvector.
- Supabase PostgreSQL storage for all persistent data.
- Threshold-based alerting mechanism.
- REST API endpoints for querying channel state, messages, sentiments, trends, costs.
- LLM cost tracking and budget monitoring.
- Structured logging and observability.

4.2 Out of Scope

- Local model hosting, GPU provisioning, or Python services.
- Frontend dashboard UI (consumers use the API or connect BI tools).
- Slack bot interactive responses (system is passive/analytical).
- Custom model fine-tuning.
- Real-time streaming analytics (batch aggregation sufficient for MVP).
- CRM or ticketing system integration (future extension).
- Supabase Auth (not needed — API is server-to-server, Slack handles its own auth).
- Supabase Storage (not needed — no file storage required).
- Supabase Realtime subscriptions (not needed for MVP — polling/API sufficient).

---

5. High-Level Architecture

5.1 Architecture Diagram (Text)

```
                    +-------------------+
                    |   Slack Platform  |
                    | (Events API +     |
                    |  Web API)         |
```

```
                +--------+----------+
                         |
                HTTP POST (signed)
                         |
                         v
                +-------------------+
                | Node.js Express   |
                | Webhook Service   |
                | (Port 3000)       |
                +--------+----------+
                         |
           +--------------+--------------+
           |                             |
      Immediate 200 OK             Async Processing
      to Slack                           |
                                         v
                              +-------------------+
                              | Supabase          |
                              | PostgreSQL        |
                              |                   |
                              | - messages table  |
                              | - channels table  |
                              | - slack_events    |
                              | - thread_edges    |
                              | - channel_state   |
                              | - message_analytics|
                              | - context_documents|
                              |   (with pgvector) |
                              | - llm_costs       |
                              | - user_profiles   |
                              | - pg-boss schema  |
                              +--------+----------+
                                       |
                              +--------+----------+
                              | pg-boss Workers   |
                              | (same Node.js     |
                              |  process)         |
                              +--------+----------+
                                       |
                        +--------------+--------------+
                        |             |             |
                        v             v             v
                +-----------+  +-----------+  +-----------+
                | Backfill  |  | Message   |  | LLM       |
                | Handler   |  | Ingest    |  | Analyze   |
                |           |  | Handler   |  | Handler   |
                +-----------+  +-----------+  +-----+-----+
                                                     |
                                                     v
                                          +-------------------+
                                          | LLM API           |
                                          | (OpenAI /         |
```

```
                                                  |  Anthropic)       |
                                                  +-------------------+
```

5.2 Data Flow Summary

```
Slack Events API
  -> Node.js Webhook Service
    -> Signature Verification (HMAC-SHA256)
      -> Idempotency Check (slack_events table INSERT ON CONFLICT)
        -> Immediate 200 OK to Slack
          -> Store Raw Message (messages table)
            -> Enqueue Job (pg-boss)
              -> Worker: Text Normalization
                -> Worker: Risk Heuristic Evaluation
                  -> IF trigger condition met:
                    -> Build Context Pack (running summary + recent messages +
pgvector retrieval)
                      -> LLM API Call (OpenAI / Anthropic)
                        -> Parse JSON Response
                          -> Validate Schema (ajv)
                            -> Store Analysis Result (message_analytics table)
                              -> Track Cost (llm_costs table)
                                -> Update Channel State (channel_state table)
                                  -> Check Alert Thresholds
                                    -> IF exceeded: Fire Alert
```

---

6. Data Flow — End-to-End

6.1 Flow 1: Slack Webhook Receipt

Trigger: Slack sends HTTP POST to `/slack/events`.

Step-by-step logic:

1. Express middleware receives the raw request body as a Buffer (required for signature verification).
2. Extract `X-Slack-Request-Timestamp` and `X-Slack-Signature` headers.
3. **Timestamp validation**: Compute `abs(now - timestamp)`. If greater than 300 seconds (5 minutes), reject with 401. This prevents replay attacks.
4. **Signature computation**: Construct basestring as `v0:{timestamp}:{rawBody}`. Compute HMAC-SHA256 using the signing secret. Prefix result with `v0=`.
5. **Timing-safe comparison**: Use `crypto.timingSafeEqual()` to compare computed signature with provided signature. This prevents timing attacks. If buffers have different lengths, reject immediately.
6. If signature is invalid, return 401.
7. Parse JSON body.
8. **URL verification challenge**: If `payload.type === 'url_verification'`, return `{ challenge: payload.challenge }` with 200. This is Slack's one-time endpoint validation.
9. **Event callback**: If `payload.type === 'event_callback'`:
    - Extract `event_id` from payload.

    - Attempt INSERT into `slack_events` table with `ON CONFLICT (workspace_id, event_id) DO NOTHING`.
    - If the insert returns 0 rows affected (conflict), this is a duplicate event. Return 200 immediately.
    - If insert succeeds, this is a new event. Continue processing.
10. Return 200 OK to Slack immediately. **All subsequent processing is asynchronous.**
11. Identify event type and dispatch:
    - `member_joined_channel` where user matches bot user ID → enqueue `channel.backfill` job.
    - `message` with valid human content (no subtype, no bot_id, has text+user+channel+ts) → enqueue `message.ingest` job.

Edge cases:

- **Slack retries**: Slack retries events up to 3 times if it doesn't receive 200 within 3 seconds. The idempotency check (slack_events table UNIQUE constraint) ensures duplicate events are silently dropped.
- **Malformed payload**: If JSON parsing fails, return 400. If event structure is unexpected, return 200 (to prevent infinite Slack retries) but log a warning.
- **Missing signing secret**: Application fails to start (enforced at config validation).
- **Clock skew**: If server clock is significantly off, all signature checks fail. Mitigation: use NTP synchronization.

6.2 Flow 2: Bot Join Detection

Trigger: `member_joined_channel` event where `event.user` matches the bot's user ID.

Step-by-step logic:

1. Extract `channel_id` from event.
2. Extract `workspace_id` (team_id) from event callback payload.
3. Upsert into `channels` table: set `status = 'initializing'`.
4. Upsert into `channel_state` table: initialize with empty running_summary, empty participants, zero messages_since_last_llm.
5. Enqueue `channel.backfill` job with payload `{ workspaceId, channelId, reason: 'member_joined_channel' }`.
6. Return 200 to Slack.

Edge cases:

- **Bot re-invited after being removed**: Channel may already exist with `status = 'ready'` or `status = 'failed'`. Set status back to `'initializing'` and re-run backfill. Existing messages from previous membership are preserved; backfill adds new ones.
- **Bot already in channel**: The `member_joined_channel` event only fires on actual join. If bot is already a member, this event won't fire. No action needed.
- **Bot user ID unknown at startup**: At application startup, call `auth.test` with the bot token to resolve the bot's own user ID. Fall back to `SLACK_BOT_USER_ID` env var if the API call fails.
- **Private channels**: Bot must be explicitly invited. It needs `groups:history` scope in addition to `channels:history`.

6.3 Flow 3: Historical Backfill

Trigger: `channel.backfill` job dequeued by pg-boss worker.

Step-by-step logic:

1. Retrieve channel record from `channels` table. Confirm status is `'initializing'`.
2. Calculate `oldest` timestamp: `Math.floor(Date.now() / 1000) - (BACKFILL_DAYS * 86400)`.
3. **Phase 1 — Fetch channel history**:
   - Call `conversations.history(channel, oldest, limit=200)`.
   - For each message in the response:
     - If it is a human message (has `user`, has `text`, no `subtype`, no `bot_id`):
       - INSERT into `messages` table with `ON CONFLICT (workspace_id, channel_id, ts) DO NOTHING`.
     - If `reply_count > 0`: add `ts` to a `threadRoots` set.
   - If `response_metadata.next_cursor` exists, save cursor and repeat.
   - Pagination continues until no more pages or `maxBackfillPages` reached.
4. **Phase 2 — Fetch thread replies**:
   - For each `threadTs` in `threadRoots`:
     - Call `conversations.replies(channel, ts=threadTs, limit=200)`.
     - For each reply message:
       - INSERT into `messages` table with idempotent upsert.
       - INSERT into `thread_edges` table `(workspace_id, channel_id, threadTs, reply.ts)` with idempotent upsert.
     - Paginate if needed.
5. **Phase 3 — Resolve user profiles**:
   - Query all distinct `user_id` values from messages for this channel.
   - For each user not already in `user_profiles` cache:
     - Call `users.info(user_id)`.
     - INSERT into `user_profiles` table.
   - Rate limit: max 5 concurrent profile fetches.
6. **Phase 4 — Generate initial context**:
   - Count total messages, distinct users, distinct threads.
   - Build basic running summary: "{N} messages from {M} participants across {T} threads over the last {D} days."
   - UPDATE `channel_state` with initial running_summary, participants_json, active_threads_json.
7. **Phase 5 — Process buffered real-time messages**:
   - During backfill, any new messages arriving via webhook were enqueued as `message.ingest` jobs.
   - These jobs will naturally be processed after the backfill completes because pg-boss processes them in order.
   - No special buffering logic needed at the queue level.

8. **Phase 6 — Finalize**:
    - UPDATE `channels` SET `status = 'ready'`, `initialized_at = NOW()`.
    - Log: `[backfill:done] channel={channelId} messages={count} threads={count} users={count}`.

Edge cases:

- **Slack API rate limiting (429)**: The `slackApiCall` wrapper already implements exponential backoff with jitter. Backfill resumes from the last cursor position after the retry-after period.
- **Channel with 100k+ messages**: Governed by `BACKFILL_MAX_PAGES` (default 100 pages * 200 messages = 20,000 messages max). For very active channels, only the most recent messages within the backfill window are captured. This is a deliberate policy decision to prevent runaway API calls.
- **Backfill job fails mid-way**: pg-boss retries the job (3 attempts, 60s backoff). Since all message inserts are idempotent (ON CONFLICT DO NOTHING), re-running backfill from scratch is safe — already-stored messages won't be duplicated.
- **Channel deleted during backfill**: Slack API returns `channel_not_found` error. Backfill handler catches this, sets channel status to `'failed'`, logs the error.
- **Bot removed during backfill**: Slack API returns `not_in_channel` error. Same handling as above.
- **Empty channel**: No messages returned. Channel transitions to `'ready'` with empty state. Perfectly valid.
- **Messages arriving during backfill**: These are stored normally via the `message.ingest` job. The webhook handler doesn't need to buffer — pg-boss queue ordering handles sequencing naturally.

6.4 Flow 4: Real-Time Message Ingestion

Trigger: `message.ingest` job dequeued by pg-boss worker.

Step-by-step logic:

1. Extract `workspaceId`, `channelId`, `ts` from job payload.
2. Retrieve the message from Supabase by `(workspace_id, channel_id, ts)`.
    - If message already exists (stored at webhook time), proceed with normalization.
    - If message doesn't exist yet (edge case), store it first.
3. **Text normalization** (see Section 12 for full pipeline):
    - Strip Slack markup (<@mentions>, <#channels>, URLs, formatting chars).
    - Convert emoji to text descriptors.
    - Preserve sentiment signals (negation, intensifiers, punctuation, caps).
    - Truncate to 4,000 characters.
    - UPDATE `messages` SET `normalized_text = {result}`.
4. **Thread edge tracking**:
    - If message has `thread_ts` and `thread_ts !== ts`:
        - INSERT into `thread_edges (workspace_id, channel_id, thread_ts, child_ts)`.
5. **LLM gate evaluation** (see Section 14 for full logic):
    - Run risk heuristic against normalized text.
    - Increment `channel_state.messages_since_last_llm`.
    - Check trigger conditions:
        - Risk score >= 0.7 → immediate trigger
        - messages_since_last_llm >= 20 → threshold trigger

        - Time since last_llm_run_at >= 10 minutes → time trigger
    - If any trigger fires AND cooldown is not active:
        - Enqueue `llm.analyze` job.
        - Reset `messages_since_last_llm = 0`.
        - SET `llm_cooldown_until = NOW() + 60 seconds`.

6. Update `channel_state.last_event_at = NOW()`.

Edge cases:

- **Duplicate message.ingest job**: pg-boss ensures at-least-once delivery. Message upsert is idempotent. Normalization is deterministic. Thread edge upsert is idempotent. LLM gate evaluation may fire an extra LLM call — acceptable because the cooldown timer prevents true rapid-fire.
- **Message with no text** (file upload, reaction, etc.): Filtered at webhook handler level. Only messages with non-empty text, valid user, no subtype, no bot_id are enqueued.
- **Very long message (>4000 chars)**: Truncated during normalization. The truncation point is after the last complete word before 4000 characters.
- **Message in unknown channel**: If channel not in `channels` table, create it with status `'pending'`. This handles the case where a message arrives before the bot join event (race condition in Slack event delivery).

6.5 Flow 5: LLM Analysis

Trigger: `llm.analyze` job dequeued by pg-boss worker.

Step-by-step logic:

1. Extract `workspaceId`, `channelId`, `triggerType`, optional `threadTs` from job payload.
2. **Assemble context pack** (see Section 15):
    - Fetch `channel_state.running_summary`.
    - Fetch `channel_state.key_decisions_json` (last 10).
    - Fetch top 3 semantically relevant `context_documents` via pgvector cosine similarity.
    - If `triggerType === 'thread'` and `threadTs` provided:
        - Fetch last 25 messages from that thread.
    - Else:
        - Fetch last 15 messages from the channel (all threads).
    - Enforce token budget: if total context exceeds 3,500 tokens, truncate oldest raw messages.
3. **Build LLM prompt**:
    - System prompt: emotion classification engine instructions with JSON schema.
    - Context section: running summary + relevant rollups.
    - Messages section: normalized text of target messages with user identifiers.
4. **Call LLM API**:
    - Select provider based on `LLM_PROVIDER` env var.
    - For OpenAI: use `openai.chat.completions.create()` with `response_format: { type: 'json_object' }`.
    - For Anthropic: use `anthropic.messages.create()` with system prompt enforcing JSON.
    - Set `temperature: 0.1` (near-deterministic for classification).
    - Set `max_tokens: 500`.
5. **Parse and validate response**:
    - Parse response text as JSON.

    - Validate against MessageAnalysis or ThreadAnalysis schema using ajv.
    - If validation fails:
        - **First attempt**: Retry with stricter prompt appending: "IMPORTANT: Return ONLY valid JSON matching the exact schema. No markdown, no code blocks, no extra text."
        - **Second attempt**: If still invalid, mark as failed. Log raw response. Do not store partial results.

6. **Store results**:
    - INSERT into `message_analytics` with all fields (dominant_emotion, confidence, escalation_risk, themes, explanation, raw_llm_response, llm_provider, llm_model, token_usage).
    - INSERT into `llm_costs` with token counts and estimated cost.
    - UPDATE `channel_state` : SET `last_llm_run_at = NOW()` , update `sentiment_snapshot_json` .
7. **Alert evaluation**:
    - If `escalation_risk === 'high'` : fire alert.
    - If `dominant_emotion === 'anger' AND confidence > 0.85` : fire alert.
    - Alert mechanism: log structured alert event. Future: webhook to Slack internal channel or PagerDuty.

Edge cases:

- **LLM API down (network error, 500)**: pg-boss retries the job (2 attempts, 30s backoff). Messages remain in queue. No data loss.
- **LLM rate limited (429)**: pg-boss retry with backoff. The retry-after header is respected if available.
- **LLM returns prose instead of JSON**: First retry with stricter prompt catches most cases. Second failure → mark as failed.
- **LLM returns valid JSON but wrong schema** (e.g., missing field, invalid emotion label): ajv validation catches this. Same retry logic as invalid JSON.
- **LLM returns confidence > 1 or < 0**: ajv schema enforces range. Treated as validation failure.
- **Empty context pack** (new channel with no history): LLM still receives the message text. Running summary is empty string. Analysis proceeds normally.
- **Token budget exceeded**: Context assembler truncates oldest messages. Running summary is never truncated. If even a single message exceeds the budget, it's truncated to 4000 chars.
- **Concurrent LLM calls for same channel**: Possible if two trigger conditions fire simultaneously. Both results are stored. Last-write-wins for channel_state update. This is acceptable — slightly more LLM cost but no data inconsistency.
- **Cost overrun**: If daily cost exceeds the configured budget threshold (checked in `llm_costs` aggregation), future LLM jobs for that workspace are paused until the next day. This is a circuit breaker pattern.

---

7. Supabase Platform Architecture

7.1 Why Supabase Over Self-Managed PostgreSQL

Dimension	Self-Managed PostgreSQL	Supabase

Provisioning	Manual: install, configure, secure	One-click project creation
Connection pooling	Deploy PgBouncer separately	Built-in Supavisor pooler
pgvector	Install extension manually	Pre-installed, ready to use
pg_cron	Install extension manually	Available on Pro plan
Backups	Manual or scripted	Automatic daily backups
Scaling	Manual vertical/horizontal	One-click compute scaling
Monitoring	Deploy Prometheus/Grafana	Built-in dashboard
SSL/TLS	Configure manually	Enforced by default
Migration path	N/A	Standard PostgreSQL — can migrate to self-hosted by changing connection string

7.2 Supabase Configuration

Required project settings:

- Region: closest to primary Slack workspace (e.g., us-east-1).
- Compute size: Small (2 vCPU, 1GB RAM) for MVP. Scale up as needed.
- Extensions to enable: `pgcrypto`, `vector`, `pg_cron`.
- Connection pooling mode: Transaction (via Supavisor).

Connection strings (from Supabase dashboard):

```
# Direct connection (for pg-boss, migrations)
DATABASE_URL=postgresql://postgres.[project-ref]:[password]@aws-0-
[region].pooler.supabase.com:5432/postgres

# Pooled connection (for application queries)
DATABASE_URL_POOLED=postgresql://postgres.[project-ref]:[password]@aws-0-
[region].pooler.supabase.com:6543/postgres?pgbouncer=true

# Supabase client
SUPABASE_URL=https://[project-ref].supabase.co
SUPABASE_SERVICE_ROLE_KEY=[service-role-key]
```

Important: pg-boss requires a direct (non-pooled) connection because it uses PostgreSQL LISTEN/NOTIFY and advisory locks, which don't work through connection poolers. Application queries use the pooled connection.

7.3 Supabase Client vs Direct PostgreSQL

For this system, we use **both**:

1. **`@supabase/supabase-js` client**: For simple CRUD operations, real-time features (future), and Supabase-specific features. Used for message queries, channel state reads, analytics.
2. **`pg` (node-postgres) with direct connection**: For pg-boss (requires LISTEN/NOTIFY), migrations (requires DDL), and complex queries (CTEs, window functions) that are cleaner in raw SQL.

This dual approach gives us the best of both worlds: Supabase's ergonomic client for simple operations, and raw SQL power for complex operations.

---

8. Database Schema Design

8.1 Design Principles

1. **Composite uniqueness on (workspace_id, channel_id, ts)**: Enables multi-workspace from day one. Every query filters by workspace_id + channel_id for isolation.
2. **Slack `ts` stored as TEXT**: Slack timestamps are string-encoded floats like `"1678901234.567890"`. They must not be stored as numeric because precision matters for identity. They serve as unique message IDs within a channel.
3. **JSONB for flexible nested data**: LLM responses, participant maps, and thread lists are stored as JSONB for flexible querying without schema changes.
4. **pgvector for semantic search**: Context documents store 1536-dimensional embeddings (OpenAI `text-embedding-3-small`) for cosine similarity retrieval.
5. **pg-boss schema separation**: pg-boss creates its own schema (`pgboss`) and manages its own tables. No manual table creation needed for the queue.

8.2 Entity Relationship Diagram (Text)

```
channels (1) ---< messages (many)
channels (1) ---< channel_state (1)
channels (1) ---< context_documents (many)
messages (1) ---< message_analytics (1)
messages (1) ---< thread_edges (many, as parent or child)
user_profiles (1) ---< messages (many, via user_id)
llm_costs (standalone, per LLM call)
slack_events (standalone, for idempotency)
```

8.3 Table: `channels`

Purpose: Registry of all channels the bot is monitoring.

```
CREATE TABLE channels (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id    TEXT NOT NULL,
    channel_id      TEXT NOT NULL,
    name            TEXT,
    status          TEXT NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending','initializing','ready','failed')),
    initialized_at  TIMESTAMPTZ,
    last_event_at   TIMESTAMPTZ,
```

```
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (workspace_id, channel_id)
);
```

Status state machine:

```
pending --> initializing --> ready
                |                |
                v                v
              failed          failed
                |                |
                v                v
          initializing      initializing  (re-invite re-triggers)
```

8.4 Table: `slack_events`

Purpose: Idempotency enforcement. Replaces the in-memory `Set<string>` that was limited to 10,000 entries and lost on restart.

```
CREATE TABLE slack_events (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id    TEXT NOT NULL,
    event_id        TEXT NOT NULL,
    event_type      TEXT NOT NULL,
    received_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (workspace_id, event_id)
);

CREATE INDEX idx_slack_events_received
    ON slack_events (received_at);
```

Retention: Rows older than 7 days are deleted by a pg_cron job. Slack retries occur within minutes, not days, so 7 days is extremely conservative.

```
-- pg_cron job (set up once)
SELECT cron.schedule(
    'cleanup-slack-events',
    '0 3 * * *',  -- daily at 3 AM
    $$DELETE FROM slack_events WHERE received_at < NOW() - INTERVAL '7 days'$$
);
```

8.5 Table: `messages`

Purpose: Raw message storage. The system of record for all Slack messages observed by the bot.

```
CREATE TABLE messages (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id    TEXT NOT NULL,
```

```
    channel_id      TEXT NOT NULL,
    ts              TEXT NOT NULL,
    thread_ts       TEXT,
    user_id         TEXT NOT NULL,
    text            TEXT NOT NULL,
    normalized_text TEXT,
    subtype         TEXT,
    bot_id          TEXT,
    source          TEXT NOT NULL DEFAULT 'realtime'
                    CHECK (source IN ('realtime','backfill')),
    analysis_status TEXT NOT NULL DEFAULT 'pending'
                    CHECK (analysis_status IN (
                        'pending','processing','completed','failed','skipped'
                    )),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (workspace_id, channel_id, ts)
);

-- For channel-scoped time-range queries
CREATE INDEX idx_messages_channel_time
    ON messages (workspace_id, channel_id, created_at);

-- For thread reconstruction queries
CREATE INDEX idx_messages_thread
    ON messages (workspace_id, channel_id, thread_ts)
    WHERE thread_ts IS NOT NULL;

-- For worker queue polling (pending analysis)
CREATE INDEX idx_messages_analysis_pending
    ON messages (analysis_status)
    WHERE analysis_status = 'pending';

-- For per-user message queries
CREATE INDEX idx_messages_user
    ON messages (workspace_id, user_id);
```

Retention: pg_cron job deletes rows older than 90 days:

```
SELECT cron.schedule(
    'cleanup-old-messages',
    '0 4 * * *',
    $$DELETE FROM messages WHERE created_at < NOW() - INTERVAL '90 days'$$
);
```

8.6 Table: `thread_edges`

Purpose: Explicit thread structure. Maps parent-child relationships for conversation graph reconstruction.

```sql
CREATE TABLE thread_edges (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id    TEXT NOT NULL,
    channel_id      TEXT NOT NULL,
    thread_ts       TEXT NOT NULL,
    child_ts        TEXT NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (workspace_id, channel_id, thread_ts, child_ts)
);

CREATE INDEX idx_thread_edges_root
    ON thread_edges (workspace_id, channel_id, thread_ts);
```

8.7 Table: `user_profiles`

Purpose: Cached Slack user profiles for display name resolution.

```sql
CREATE TABLE user_profiles (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id    TEXT NOT NULL,
    user_id         TEXT NOT NULL,
    display_name    TEXT,
    real_name       TEXT,
    profile_image   TEXT,
    fetched_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (workspace_id, user_id)
);
```

Cache TTL: Profiles older than 24 hours are refreshed on next access. The application maintains an in-memory Map as a hot cache, with Supabase as the durable backing store.

8.8 Table: `channel_state`

Purpose: Per-channel derived state. One row per channel. Contains running summary, participants, active threads, key decisions, sentiment snapshot, and LLM gating state.

```sql
CREATE TABLE channel_state (
    id                          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id                TEXT NOT NULL,
    channel_id                  TEXT NOT NULL,
    running_summary             TEXT NOT NULL DEFAULT '',
    participants_json           JSONB DEFAULT '{}',
    active_threads_json         JSONB DEFAULT '[]',
    key_decisions_json          JSONB DEFAULT '[]',
    sentiment_snapshot_json     JSONB DEFAULT '{}',
    messages_since_last_llm     INTEGER NOT NULL DEFAULT 0,
    last_llm_run_at             TIMESTAMPTZ,
    llm_cooldown_until          TIMESTAMPTZ,
    last_reconcile_at           TIMESTAMPTZ,
```

```
    updated_at                  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (workspace_id, channel_id)
);
```

JSON structure examples:

```
// participants_json
{
    "U12345": { "displayName": "Alice", "messageCount": 47 },
    "U67890": { "displayName": "Bob", "messageCount": 23 }
}

// active_threads_json
[
    { "threadTs": "1678901234.567890", "messageCount": 12, "lastActivityAt": "2026-
03-04T10:00:00Z" }
]

// key_decisions_json
[
    { "text": "Agreed to use Supabase for backend", "ts": "1678901234.567890",
"detectedAt": "2026-03-04T10:00:00Z" }
]

// sentiment_snapshot_json
{
    "totalAnalyzed": 150,
    "emotionDistribution": { "joy": 45, "neutral": 80, "anger": 10, "sadness": 15 },
    "averageConfidence": 0.78,
    "highRiskCount": 3,
    "lastUpdated": "2026-03-04T10:00:00Z"
}
```

8.9 Table: `message_analytics`

Purpose: Structured LLM analysis results for individual messages or message batches.

```
CREATE TABLE message_analytics (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id        TEXT NOT NULL,
    channel_id          TEXT NOT NULL,
    message_ts          TEXT NOT NULL,
    dominant_emotion    TEXT NOT NULL
                        CHECK (dominant_emotion IN (

'anger','disgust','fear','joy','neutral','sadness','surprise'
                        )),
    confidence          REAL NOT NULL
                        CHECK (confidence >= 0 AND confidence <= 1),
    escalation_risk     TEXT NOT NULL
```

```
                        CHECK (escalation_risk IN ('low','medium','high')),
    themes              JSONB DEFAULT '[]',
    decision_signal     BOOLEAN DEFAULT FALSE,
    explanation         TEXT,
    raw_llm_response    JSONB NOT NULL,
    llm_provider        TEXT NOT NULL,
    llm_model           TEXT NOT NULL,
    token_usage         JSONB,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (workspace_id, channel_id, message_ts)
);

CREATE INDEX idx_analytics_channel_time
    ON message_analytics (workspace_id, channel_id, created_at);

CREATE INDEX idx_analytics_emotion
    ON message_analytics (dominant_emotion);

CREATE INDEX idx_analytics_high_risk
    ON message_analytics (escalation_risk)
    WHERE escalation_risk IN ('medium','high');
```

8.10 Table: `context_documents`

Purpose: Rollup summaries and vector embeddings for semantic retrieval. This is how the system maintains long-term memory without sending entire message histories to the LLM.

```
CREATE TABLE context_documents (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id    TEXT NOT NULL,
    channel_id      TEXT NOT NULL,
    doc_type        TEXT NOT NULL
                    CHECK (doc_type IN
('channel_rollup','thread_rollup','decision_log')),
    content         TEXT NOT NULL,
    embedding       vector(1536),
    window_start    TIMESTAMPTZ,
    window_end      TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_context_docs_channel
    ON context_documents (workspace_id, channel_id, doc_type);

-- IVFFlat index for approximate nearest neighbor search
-- lists=100 is appropriate for up to ~100K documents per channel
CREATE INDEX idx_context_docs_embedding
    ON context_documents
    USING ivfflat (embedding vector_cosine_ops)
    WITH (lists = 100);
```

Semantic retrieval query:

```
SELECT id, content, 1 - (embedding <=> $1::vector) AS similarity
FROM context_documents
WHERE workspace_id = $2
  AND channel_id = $3
ORDER BY embedding <=> $1::vector
LIMIT 3;
```

8.11 Table: `llm_costs`

Purpose: Per-request LLM cost tracking for budget monitoring and cost attribution.

```
CREATE TABLE llm_costs (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id        TEXT NOT NULL,
    channel_id          TEXT,
    llm_provider        TEXT NOT NULL,
    llm_model           TEXT NOT NULL,
    prompt_tokens       INTEGER NOT NULL,
    completion_tokens   INTEGER NOT NULL,
    estimated_cost_usd  DECIMAL(10,6) NOT NULL,
    job_type            TEXT,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_llm_costs_workspace_time
    ON llm_costs (workspace_id, created_at);
```

Cost calculation logic:

```
GPT-4o-mini: $0.15 / 1M input tokens, $0.60 / 1M output tokens
GPT-4o:      $2.50 / 1M input tokens, $10.00 / 1M output tokens
Claude Haiku: $0.25 / 1M input tokens, $1.25 / 1M output tokens
Claude Sonnet: $3.00 / 1M input tokens, $15.00 / 1M output tokens
```

Daily cost aggregation query:

```
SELECT DATE(created_at) AS day,
       SUM(estimated_cost_usd) AS total_cost,
       SUM(prompt_tokens) AS total_prompt_tokens,
       SUM(completion_tokens) AS total_completion_tokens
FROM llm_costs
WHERE workspace_id = $1
  AND created_at >= NOW() - INTERVAL '30 days'
GROUP BY DATE(created_at)
ORDER BY day DESC;
```

---

9. Event Ingestion Pipeline

9.1 Webhook Handler Architecture

The webhook handler is the system's front door. It must be fast (return 200 within 3 seconds), secure (verify every signature), and idempotent (reject duplicates).

```
HTTP POST /slack/events
  |
  v
[express.raw({ type: 'application/json' })]  <-- Captures raw body for HMAC
  |
  v
[isValidSlackSignature(req, rawBody)]  <-- HMAC-SHA256 verification
  |
  +-- FAIL --> 401 Unauthorized
  |
  v
[JSON.parse(rawBody)]
  |
  +-- FAIL --> 400 Bad Request
  |
  v
[url_verification?]  --> YES --> Return { challenge } --> DONE
  |
  NO
  |
  v
[event_callback?]  --> NO --> 200 OK (ignore unknown types)
  |
  YES
  |
  v
[INSERT INTO slack_events ON CONFLICT DO NOTHING]
  |
  +-- 0 rows (duplicate) --> 200 OK --> DONE
  |
  1 row (new event)
  |
  v
[200 OK to Slack]  <-- IMMEDIATELY, before any processing
  |
  v
[Identify event type]
  |
  +-- member_joined_channel (bot) --> Enqueue channel.backfill
  |
  +-- message (human) --> Store in messages table + Enqueue message.ingest
  |
  +-- other --> Log, ignore
```

9.2 Idempotency Guarantee

Layer 1 — Database UNIQUE constraint: `slack_events (workspace_id, event_id)`. This is the authoritative deduplication layer. INSERT ON CONFLICT DO NOTHING is atomic and race-free.

Why not Redis-based fast path?: In v1 with Supabase, we eliminate Redis entirely. The Supabase PostgreSQL INSERT ON CONFLICT is fast enough (< 5ms for indexed lookups). If latency becomes an issue at scale, a Redis SET NX cache can be added as an optimization layer in v2.

Why not in-memory Set?: The current codebase uses an in-memory Set with 10,000 entry limit. This is lost on restart and has unbounded growth potential. The database approach is durable and bounded by the 7-day retention cron job.

---

10. Backfill Engine

10.1 Backfill Algorithm

```
FUNCTION runBackfill(workspaceId, channelId, reason):
    channel = getChannel(workspaceId, channelId)
    IF channel.status NOT IN ('pending', 'initializing', 'failed'):
        LOG warning "Channel already ready, skipping backfill"
        RETURN

    SET channel.status = 'initializing'
    oldest = NOW() - BACKFILL_DAYS

    // Phase 1: Fetch history
    threadRoots = Set()
    cursor = null
    pageCount = 0

    LOOP:
        IF pageCount >= MAX_BACKFILL_PAGES: BREAK

        response = slackApiCall('conversations.history', {
            channel: channelId,
            oldest: oldest,
            cursor: cursor,
            limit: SLACK_PAGE_SIZE
        })

        FOR EACH message IN response.messages:
            IF isHumanMessage(message):
                upsertMessage(workspaceId, channelId, message, 'backfill')

            IF message.reply_count > 0:
                threadRoots.add(message.ts)

        cursor = response.response_metadata?.next_cursor
        IF NOT cursor: BREAK
```

```
        pageCount++

    // Phase 2: Fetch thread replies
    FOR EACH rootTs IN threadRoots:
        replies = fetchAllThreadReplies(channelId, rootTs)

        FOR EACH reply IN replies:
            IF isHumanMessage(reply):
                upsertMessage(workspaceId, channelId, reply, 'backfill')
                upsertThreadEdge(workspaceId, channelId, rootTs, reply.ts)

    // Phase 3: Resolve user profiles
    userIds = getDistinctUserIds(workspaceId, channelId)
    resolveUserProfiles(workspaceId, userIds)

    // Phase 4: Initialize channel state
    refreshChannelState(workspaceId, channelId)

    // Phase 5: Mark ready
    SET channel.status = 'ready'
    SET channel.initialized_at = NOW()
```

10.2 Pagination Mechanics

Slack's `conversations.history` returns messages in **reverse chronological order** (newest first). Each page contains up to `limit` messages (max 1000, default 200). The `cursor` token enables forward pagination.

```
Page 1: [msg_newest, msg_n-1, ..., msg_n-199]  cursor: "abc123"
Page 2: [msg_n-200, msg_n-201, ..., msg_n-399]  cursor: "def456"
...
Page N: [msg_oldest+1, msg_oldest]  cursor: null (no more)
```

Maximum messages per backfill: `BACKFILL_MAX_PAGES * SLACK_PAGE_SIZE` = 100 * 200 = 20,000 messages (default configuration).

10.3 Thread Fetch Mechanics

For each message where `reply_count > 0`, we call `conversations.replies(channel, ts)`. This returns ALL messages in the thread (including the root message) in chronological order.

```
conversations.replies(channel="C123", ts="1678901234.567890")
Response:
  [
    { ts: "1678901234.567890", text: "Original question", thread_ts:
"1678901234.567890" },  // root
    { ts: "1678901300.000000", text: "First reply", thread_ts: "1678901234.567890" },
    { ts: "1678901400.000000", text: "Second reply", thread_ts: "1678901234.567890" }
  ]
```

Key observation: The root message appears in BOTH `conversations.history` and `conversations.replies`. Our idempotent upsert handles this — the duplicate insert is silently ignored.

---

11. Thread Reconstruction

11.1 Thread Data Model

A Slack thread is identified by `thread_ts` — the timestamp of the root message. Every reply has `thread_ts` set to the root's `ts`. The root message itself has `thread_ts === ts` (self-referencing).

```
Channel Messages (flat, from conversations.history):
  msg_A  ts=100  thread_ts=null     (standalone message)
  msg_B  ts=200  thread_ts=200      (thread root, has replies)
  msg_C  ts=300  thread_ts=null     (standalone message)

Thread B Replies (from conversations.replies):
  msg_B   ts=200  thread_ts=200     (root, already stored)
  msg_B1  ts=210  thread_ts=200     (reply)
  msg_B2  ts=220  thread_ts=200     (reply)
  msg_B3  ts=230  thread_ts=200     (reply)

Resulting thread_edges:
  (channel, thread_ts=200, child_ts=210)
  (channel, thread_ts=200, child_ts=220)
  (channel, thread_ts=200, child_ts=230)
```

11.2 Thread Graph Query

To reconstruct a complete thread for display or LLM context:

```
SELECT m.ts, m.user_id, m.text, m.normalized_text, m.created_at,
       up.display_name, up.real_name
FROM messages m
LEFT JOIN user_profiles up ON up.workspace_id = m.workspace_id AND up.user_id =
m.user_id
WHERE m.workspace_id = $1
  AND m.channel_id = $2
  AND (m.thread_ts = $3 OR m.ts = $3)
ORDER BY m.ts ASC;
```

11.3 Active Thread Detection

A thread is "active" if it received a reply within the last 24 hours. Active threads are tracked in `channel_state.active_threads_json` and are candidates for periodic reconciliation.

```
SELECT DISTINCT thread_ts,
       COUNT(*) AS reply_count,
       MAX(created_at) AS last_activity
```

```sql
FROM messages
WHERE workspace_id = $1
  AND channel_id = $2
  AND thread_ts IS NOT NULL
  AND created_at > NOW() - INTERVAL '24 hours'
GROUP BY thread_ts
ORDER BY last_activity DESC;
```

---

12. Text Normalization Pipeline

12.1 Why Normalize?

Slack messages contain markup, emoji, and formatting that is irrelevant or misleading for sentiment analysis. However, certain text patterns (capitalization, punctuation, negation) are **critical sentiment signals** that must be preserved.

12.2 Normalization Steps (Exact Order)

Step 1: Strip user mentions

```
Input:  "Hey <@U12345> can you fix this?"
Output: "Hey can you fix this?"
Regex:  /<@[A-Z0-9]+>/g → ''
```

Step 2: Resolve channel links

```
Input:  "Please post in <#C12345|general>"
Output: "Please post in #general"
Regex:  /<#[A-Z0-9]+\|([^>]+)>/g → '#$1'
```

Step 3: Clean URLs

```
Input:  "Check this <https://example.com|example.com>"
Output: "Check this [link]"
Regex:  /<(https?:\/\/[^>|]+)(\|[^>]+)?>/g → '[link]'
```

Step 4: Strip formatting characters

```
Input:  "This is *bold* and ~strikethrough~ and `code`"
Output: "This is bold and strikethrough and code"
Regex:  Remove wrapping *, ~, ` but preserve content
```

Step 5: Convert emoji to text

```
Input:  "Great work! 😊👍"
Output: "Great work! :smiling_face_with_smiling_eyes: :thumbs_up:"
Library: node-emoji
```

Step 6: Preserve sentiment signals (NO-OP — do not touch)

```
Preserved: "I am NOT happy!!!" → kept as-is
Preserved: "This is ABSOLUTELY TERRIBLE" → kept as-is (caps preserved)
Preserved: "don't, can't, won't, never" → kept as-is (negation preserved)
Preserved: "very, extremely, incredibly" → kept as-is (intensifiers preserved)
Preserved: "!!!", "???", "..." → kept as-is (punctuation patterns preserved)
```

Step 7: Collapse whitespace

```
Input:  "Too   many     spaces"
Output: "Too many spaces"
Regex:  /\s+/g → ' '
```

Step 8: Truncate

```
Max length: 4,000 characters
Truncation: At the last complete word before 4,000 chars
Suffix: None (truncation is invisible to the LLM)
```

12.3 What We Do NOT Do

- **Do NOT lowercase**: The LLM uses casing as a signal. "ANGRY" conveys more intensity than "angry".
- **Do NOT stem/lemmatize**: The LLM handles morphological variation natively.
- **Do NOT remove stopwords**: "not", "no", "never" are critical negation markers.
- **Do NOT remove punctuation**: "!!!" and "???" convey emotional intensity.

---

13. LLM Inference Design

13.1 Provider Abstraction

```typescript
interface LLMProvider {
    name: 'openai' | 'anthropic';
    analyzeMessage(text: string, context: ContextPack): Promise<LLMResult>;
    analyzeThread(messages: ThreadMessage[], context: ContextPack):
Promise<LLMResult>;
    generateEmbedding(text: string): Promise<number[]>;
}
```

Provider selection: `LLM_PROVIDER` environment variable. Default: `openai`. Changing this env var and restarting the worker switches the provider. Zero code changes needed.

13.2 Prompt Templates

Single-Message System Prompt

```
You are an emotion classification engine. Analyze the CLIENT's emotional tone
from the Slack message provided.
```

```
Context about the conversation:
{running_summary}

Recent key decisions:
{key_decisions}

Relevant historical context:
{context_documents}

Return strictly valid JSON with the following fields:
{
    "dominant_emotion": one of
["anger","disgust","fear","joy","neutral","sadness","surprise"],
    "confidence": number between 0 and 1,
    "escalation_risk": one of ["low","medium","high"],
    "explanation": string (one sentence explaining your classification)
}

Do not include any text outside the JSON object.
Do not wrap in code blocks or markdown.
Do not add commentary.
Return ONLY the JSON object.
```

Thread-Level System Prompt

Extends the single-message prompt with additional instructions:

```
Analyze the overall emotional trajectory of this conversation thread.
The messages are ordered chronologically with user identifiers.

In addition to the standard fields, also return:
{
    "dominant_emotion": ...,
    "confidence": ...,
    "escalation_risk": ...,
    "explanation": ...,
    "thread_sentiment": string (one sentence overall assessment),
    "sentiment_trajectory": one of ["improving","stable","deteriorating"],
    "summary": string (2-3 sentence thread summary)
}
```

13.3 JSON Schema Validation

```
{
    "type": "object",
    "required": ["dominant_emotion", "confidence", "escalation_risk",
"explanation"],
    "additionalProperties": false,
    "properties": {
        "dominant_emotion": {
            "type": "string",
```

```
            "enum": ["anger", "disgust", "fear", "joy", "neutral", "sadness",
"surprise"]
        },
        "confidence": {
            "type": "number",
            "minimum": 0,
            "maximum": 1
        },
        "escalation_risk": {
            "type": "string",
            "enum": ["low", "medium", "high"]
        },
        "explanation": {
            "type": "string",
            "minLength": 1,
            "maxLength": 500
        }
    }
}
```

13.4 Confidence Score Caveat

Critical distinction: LLM-generated confidence values (0-1) are **self-estimated**, not calibrated probabilities like softmax output from a trained classifier. A confidence of 0.85 from GPT-4o-mini is the model's own estimate of certainty, not a statistically calibrated probability.

Impact: For threshold-based alerting and trend analysis, this is acceptable. For statistical modeling or precise probability estimation, additional calibration would be needed (out of scope).

---

14. Conditional LLM Gating

14.1 Why Not Per-Message?

Calling the LLM for every single message is:

- **Expensive**: At $0.01-$0.03 per call, 10,000 messages/day costs $100-$300/day.
- **Unnecessary**: Most messages are neutral ("sounds good", "ok", "let me check").
- **Wasteful**: The LLM provides more value when given context (batch of messages) than individual messages.

14.2 Trigger Conditions

The LLM is triggered when ANY of these conditions is true for a channel:

```
Condition 1: RISK_HEURISTIC_SCORE >= 0.7
    → Immediate trigger (high-risk message detected)

Condition 2: messages_since_last_llm >= LLM_MSG_THRESHOLD (default: 20)
    → Threshold trigger (enough messages accumulated)
```

```
Condition 3: NOW() - last_llm_run_at >= LLM_TIME_THRESHOLD (default: 10 min)
    AND messages_since_last_llm > 0
    → Time trigger (enough time passed, messages exist)

Condition 4: Manual POST /api/channels/:channelId/analyze
    → Manual trigger (user requested)
```

Cooldown: After any automatic trigger, `llm_cooldown_until` is set to `NOW() + 60 seconds`. No automatic triggers fire during cooldown. Manual triggers bypass cooldown.

14.3 Risk Heuristic Algorithm

```
FUNCTION computeRiskScore(normalizedText: string): number
    score = 0.0

    // Keyword detection (case-insensitive)
    highRiskWords = [
        'angry', 'furious', 'frustrated', 'unacceptable',
        'terrible', 'horrible', 'ridiculous', 'escalate',
        'cancel', 'lawsuit', 'disappointed', 'outraged',
        'incompetent', 'waste of time', 'fed up'
    ]

    matchCount = 0
    FOR EACH word IN highRiskWords:
        IF normalizedText.toLowerCase().includes(word):
            matchCount++

    // Each keyword match adds 0.3, capped at 3 matches
    score += Math.min(matchCount, 3) * 0.3

    // ALL CAPS detection (sentence-level)
    sentences = normalizedText.split(/[.!?]+/)
    capsCount = sentences.filter(s => s.trim().length > 5 AND s ===
s.toUpperCase()).length
    IF capsCount > 0: score += 0.2

    // Excessive punctuation (!!!, ???, multiple exclamation/question marks)
    IF /[!]{3,}/.test(normalizedText): score += 0.15
    IF /[?]{3,}/.test(normalizedText): score += 0.15

    RETURN Math.min(score, 1.0)
```

Examples:

```
"Sounds good, thanks!" → score: 0.0 (no triggers)
"I'm a bit frustrated with the delay" → score: 0.3 (one keyword)
"This is ABSOLUTELY TERRIBLE!!! I want to CANCEL" → score: 0.3 + 0.3 + 0.2 + 0.15 =
0.95 (immediate trigger)
"I'm frustrated and disappointed" → score: 0.3 + 0.3 = 0.6 (below threshold, no
immediate trigger)
```

14.4 Gating State Machine

```
Per channel:

  [Message arrives]
       |
       v
  [Compute risk score]
       |
       +-- score >= 0.7 AND NOT cooldown_active --> TRIGGER LLM
       |
       v
  [Increment messages_since_last_llm]
       |
       +-- count >= 20 AND NOT cooldown_active --> TRIGGER LLM
       |
       v
  [Check time since last LLM]
       |
       +-- elapsed >= 10min AND count > 0 AND NOT cooldown_active --> TRIGGER LLM
       |
       v
  [No trigger] --> Done

  [TRIGGER LLM]:
       |
       v
  [Enqueue llm.analyze job]
  [Reset messages_since_last_llm = 0]
  [Set cooldown for 60s]
```

---

15. Context Management and Memory

15.1 The Context Problem

An LLM has a finite context window (128K tokens for GPT-4o, but cost scales with token count). A Slack channel may have tens of thousands of messages. We cannot send everything. We must compress history into a structured context pack that gives the LLM enough information to reason accurately.

15.2 Context Pack Structure

When the LLM is triggered, the context assembler builds a pack with these components:

```
Context Pack:
├── Running Summary (~200 tokens)          [NEVER truncated]
├── Key Decisions (last 10) (~100 tokens)  [Truncated if >10]
├── Relevant Rollups (top 3) (~600 tokens) [pgvector similarity search]
├── Target Messages (~1000 tokens)
│   ├── IF thread mode: last 25 thread messages
```

```
│   └── IF channel mode: last 15 channel messages
└── Active Thread Summaries (~200 tokens)  [Truncated oldest first]

Total budget: ~3,500 tokens (prompt)
Reserved for response: ~500 tokens
```

15.3 Token Budget Enforcement

```
FUNCTION assembleContextPack(workspaceId, channelId, mode, threadTs?):
    budget = 3500  // tokens
    pack = {}

    // Layer 1: Running summary (always included)
    summary = getChannelState(workspaceId, channelId).running_summary
    pack.summary = summary
    budget -= estimateTokens(summary)

    // Layer 2: Key decisions
    decisions = getChannelState(workspaceId, channelId).key_decisions
    decisionsText = decisions.slice(-10).join('\n')
    budget -= estimateTokens(decisionsText)
    pack.decisions = decisionsText

    // Layer 3: Relevant rollups (pgvector)
    IF budget > 600:
        rollups = queryRelevantDocuments(workspaceId, channelId, targetText, limit=3)
        rollupText = rollups.map(r => r.content).join('\n\n')
        IF estimateTokens(rollupText) <= budget * 0.3:
            pack.rollups = rollupText
            budget -= estimateTokens(rollupText)

    // Layer 4: Target messages
    IF mode === 'thread' AND threadTs:
        messages = getThreadMessages(workspaceId, channelId, threadTs, limit=25)
    ELSE:
        messages = getRecentMessages(workspaceId, channelId, limit=15)

    // Truncate messages to fit budget
    WHILE estimateTokens(formatMessages(messages)) > budget AND messages.length > 1:
        messages.shift()  // Remove oldest

    pack.messages = messages

    RETURN pack

FUNCTION estimateTokens(text: string): number
    // Rough estimate: 1 token ≈ 4 characters for English text
    RETURN Math.ceil(text.length / 4)
```

15.4 pgvector Semantic Retrieval

When building context packs, we retrieve the most semantically relevant historical summaries using vector cosine similarity:

```
SELECT content,
       1 - (embedding <=> $1::vector) AS similarity
FROM context_documents
WHERE workspace_id = $2
  AND channel_id = $3
ORDER BY embedding <=> $1::vector
LIMIT 3;
```

The query embedding `$1` is generated from the target message(s) text using OpenAI's `text-embedding-3-small` model (1536 dimensions, ~$0.00002 per embedding).

---

16. Incremental Summarization

16.1 Summarization Triggers

Trigger	Condition	Action
Channel rollup	20 new messages OR 10 minutes since last rollup	Summarize recent messages, merge with running summary
Thread rollup	10 new thread replies OR 15 minutes idle on active thread	Summarize thread, store as context document
Backfill init	Channel backfill completes	Hierarchical batch summarization of all history

16.2 Channel Rollup Process

```
1. Fetch messages since last rollup window.
2. Send to LLM:
   "Summarize the following Slack messages into a concise paragraph.
    Preserve: key decisions, action items, sentiment shifts, participant roles.
    Keep it under 100 words."
3. Receive summary text.
4. Merge with existing running_summary:
   "Given the existing channel summary and a new update, produce a consolidated
    running summary under 200 words. Preserve all key decisions and participant
context."
5. Generate embedding for the new summary via text-embedding-3-small.
6. Store in context_documents with doc_type='channel_rollup'.
7. Update channel_state.running_summary.
```

16.3 Backfill Summarization (Hierarchical Compression)

For channels with thousands of historical messages, we use hierarchical compression:

```
Level 0: Raw messages (e.g., 5000 messages)
         Split into batches of 200

Level 1: Batch summaries (e.g., 25 summaries, each ~100 words)
         Send each batch to LLM for summarization

Level 2: Meta-summaries (e.g., 5 summaries, each ~150 words)
         Group Level 1 summaries into groups of 5
         Summarize each group

Level 3: Final summary (~200 words)
         Summarize all Level 2 summaries into one
         This becomes the initial running_summary
```

Cost estimation: For 5000 messages:

- Level 1: 25 LLM calls * ~$0.02 = $0.50
- Level 2: 5 LLM calls * ~$0.02 = $0.10
- Level 3: 1 LLM call * ~$0.02 = $0.02
- Total: ~$0.62 for initial summarization

---

17. User Profile Enrichment

17.1 Resolution Pipeline

```
[Message with user_id] --> [Check in-memory cache]
    |                           |
    +-- HIT (< 24h old)  --> Use cached display name
    |
    +-- MISS or STALE --> [Check Supabase user_profiles table]
                                |
                                +-- HIT (< 24h old) --> Cache in memory + use
                                |
                                +-- MISS or STALE --> [Call Slack users.info API]
                                                            |
                                                            v
                                                      [Upsert Supabase]
                                                      [Cache in memory]
                                                      [Use display name]
```

17.2 Batch Resolution During Backfill

During backfill, we collect all unique user IDs and resolve them in batch with controlled concurrency:

```
userIds = getDistinctUserIds(messages)
unresolvedIds = userIds.filter(id => NOT inCache(id))

// Resolve with max 5 concurrent API calls
semaphore = Semaphore(5)
```

```
FOR EACH userId IN unresolvedIds:
    await semaphore.acquire()
    resolveUser(userId).finally(() => semaphore.release())
```

17.3 Slack Rate Limit Handling

`users.info` is Tier 4 (100+ requests per minute). For burst resolution during backfill, 5 concurrent calls with a 200ms delay between batches stays well within limits.

---

18. Alerting Service

18.1 Alert Conditions

Condition	Threshold	Action
High escalation risk	`escalation_risk === 'high'`	Fire alert
High anger confidence	`dominant_emotion === 'anger' AND confidence > 0.85`	Fire alert
Deteriorating thread	`sentiment_trajectory === 'deteriorating' AND escalation_risk !== 'low'`	Fire alert
Cost overrun	Daily LLM cost > configured budget	Pause LLM calls, fire alert

18.2 Alert Delivery (MVP)

For MVP, alerts are:

1. Logged as structured JSON events with severity `'alert'`.
2. Stored in a lightweight `alerts` log (or just query `message_analytics` for high-risk entries).

Future: Webhook to internal Slack channel, PagerDuty, Opsgenie integration.

---

19. Analytics REST API

19.1 Endpoint Catalog

Core Endpoints

Method	Path	Description
GET	`/`	Health check (version, uptime, queue stats)
GET	`/api/channels/:channelId/state`	Channel state (summary, participants, threads, sentiment)
GET	`/api/channels/:channelId/messages`	Paginated messages (query: limit, threadTs, from, to)

GET	`/api/channels/:channelId/threads`	Active threads with reply counts
GET	`/api/channels/:channelId/timeline`	Sentiment timeline (query: from, to, granularity)
POST	`/api/channels/:channelId/backfill`	Manual backfill trigger
POST	`/api/channels/:channelId/analyze`	Manual LLM analysis trigger

Analytics Endpoints

Method	Path	Description
GET	`/api/sentiments`	Aggregated emotion distribution (query: channel_id, from, to)
GET	`/api/sentiments/:messageTs`	Single message sentiment result
GET	`/api/trends`	Time-series emotion trends (query: workspace_id, granularity)
GET	`/api/costs`	LLM cost tracking (query: workspace_id, from, to)

Webhook Endpoint

Method	Path	Description
POST	`/slack/events`	Slack Events API webhook

19.2 Response Formats

GET /api/channels/:channelId/state

```
{
    "channelId": "C12345",
    "status": "ready",
    "initializedAt": "2026-03-04T10:00:00Z",
    "runningSummary": "Active discussion about API redesign...",
    "participants": [
        { "userId": "U12345", "displayName": "Alice", "messageCount": 47 }
    ],
    "activeThreads": [
        { "threadTs": "1678901234.567890", "messageCount": 12, "lastActivityAt":
"..." }
    ],
    "sentimentSnapshot": {
        "totalAnalyzed": 150,
        "emotionDistribution": { "joy": 45, "neutral": 80, "anger": 10 },
        "highRiskCount": 3
    },
    "messageCount": 1234,
    "lastEventAt": "2026-03-04T15:30:00Z"
}
```

20. Queue Architecture (pg-boss over Supabase)

20.1 Why pg-boss?

pg-boss uses PostgreSQL as its backing store (SKIP LOCKED for job claiming, LISTEN/NOTIFY for instant job pickup). Since we already have Supabase PostgreSQL, pg-boss adds zero infrastructure.

Important: pg-boss requires a **direct** (non-pooled) PostgreSQL connection because it uses LISTEN/NOTIFY and advisory locks. Supabase provides both pooled (port 6543) and direct (port 5432) connections. pg-boss uses the direct connection.

20.2 Job Definitions

```
// Job type payloads
interface BackfillJob {
    workspaceId: string;
    channelId: string;
    reason: string;
}

interface MessageIngestJob {
    workspaceId: string;
    channelId: string;
    ts: string;
    eventId: string;
}

interface LLMAnalyzeJob {
    workspaceId: string;
    channelId: string;
    triggerType: 'risk' | 'threshold' | 'time' | 'manual';
    threadTs?: string;
}

interface SummaryRollupJob {
    workspaceId: string;
    channelId: string;
    windowStart: string;
    windowEnd: string;
}
```

20.3 Concurrency Configuration

```
const queueConfig = {
    'channel.backfill':     { teamConcurrency: 2,  retryLimit: 3, retryDelay: 60 },
    'message.ingest':       { teamConcurrency: 8,  retryLimit: 3, retryDelay: 10 },
    'llm.analyze':          { teamConcurrency: 4,  retryLimit: 2, retryDelay: 30 },
```

```
    'summary.rollup':       { teamConcurrency: 3,  retryLimit: 2, retryDelay: 30 },
    'thread.reconcile':     { teamConcurrency: 3,  retryLimit: 2, retryDelay: 15 },
    'user.resolve':         { teamConcurrency: 5,  retryLimit: 3, retryDelay: 5 },
};
```

---

21. Concurrency Model

21.1 Single-Process Architecture

For MVP (~50 active channels), the system runs as a single Node.js process:

- Express HTTP server handles webhooks and API requests.
- pg-boss workers run in the same process (event loop based, not blocking).
- Supabase client manages connection pooling.

21.2 Race Conditions and Mitigations

Race Condition	Scenario	Mitigation
Duplicate events	Slack sends same event twice simultaneously	`slack_events` UNIQUE constraint — INSERT ON CONFLICT DO NOTHING
Duplicate messages	Same message stored during backfill and realtime	`messages` UNIQUE on (workspace_id, channel_id, ts)
Concurrent channel_state updates	Two workers update same channel state	Use `UPDATE ... WHERE` with optimistic concurrency or pg advisory locks
Backfill + realtime overlap	New messages arrive during backfill	pg-boss ordering — message.ingest jobs processed after backfill job completes
Multiple LLM triggers	Risk trigger and threshold trigger fire simultaneously	Cooldown timer prevents rapid-fire; duplicate analysis is acceptable (idempotent storage)

---

22. Security Architecture

22.1 Authentication and Verification

- Every Slack webhook request verified via HMAC-SHA256 with timing-safe comparison.
- Slack Bot Token stored in env vars, never logged or exposed.
- LLM API keys stored in env vars, never logged or exposed.
- Supabase service role key stored in env vars (server-side only).
- Analytics API: no auth for MVP. Future: API key auth or Supabase Auth.

22.2 Data Protection

- Supabase enforces SSL/TLS for all connections.
- Database encrypted at rest (Supabase managed).

- Raw message text stored in database for audit but NEVER in application logs.
- LLM data privacy: Messages sent to OpenAI/Anthropic APIs are NOT used for training (per their commercial API policies).

22.3 Channel Isolation

Every database query includes `(workspace_id, channel_id)` filters. There is no query path that can return data across channels. Row Level Security (RLS) can be enabled on Supabase as an additional enforcement layer:

```
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;
CREATE POLICY "channel_isolation" ON messages
    USING (workspace_id = current_setting('app.workspace_id')
       AND channel_id = current_setting('app.channel_id'));
```

22.4 Secret Management

```
REQUIRED (fail to start if missing):
  SLACK_SIGNING_SECRET
  DATABASE_URL

REQUIRED FOR FUNCTIONALITY:
  SLACK_BOT_TOKEN (backfill disabled without it)

REQUIRED FOR LLM (Phase C):
  OPENAI_API_KEY or ANTHROPIC_API_KEY

OPTIONAL:
  SLACK_BOT_USER_ID (auto-detected via auth.test)
  SUPABASE_URL + SUPABASE_SERVICE_ROLE_KEY (for Supabase JS client)
```

---

23. Edge Cases and Failure Modes

23.1 Comprehensive Edge Case Catalog

Slack API Edge Cases

#	Edge Case	Behavior	Handling
E01	Slack sends event with no event_id	Cannot deduplicate	Process anyway; log warning; extremely rare
E02	Slack retries after our 200 (network issue on their end)	Duplicate delivery	Idempotent upsert in slack_events handles it
E03	Bot invited to private channel without groups:history scope	Cannot read history	Backfill fails with scope error; log; set channel status='failed'

E04	Channel is archived after bot join	Cannot read or write	API returns channel_is_archived; mark channel failed
E05	User mentions a deleted user	user_id exists but users.info fails	Store message with userId, mark profile as unknown
E06	Message edited after storage	We stored old version	We do NOT handle edits in MVP; documented limitation
E07	Message deleted after storage	We stored deleted message	We do NOT handle deletions in MVP; documented limitation
E08	Slack free plan limits history	conversations.history returns partial	Backfill stores what's available; no error
E09	Bot token rotated	All API calls fail	App health check fails; alerts fire; requires env update and restart
E10	Signing secret rotated	All signature verifications fail	Same as above

LLM Edge Cases

#	Edge Case	Behavior	Handling
E11	LLM returns empty string	No analysis possible	Treat as invalid response; retry once; mark failed
E12	LLM returns JSON wrapped in markdown code fence	Parser extracts JSON from fence	Strip ``` markers before JSON.parse
E13	LLM invents emotion not in enum	ajv validation fails	Retry with stricter prompt; mark failed on second attempt
E14	LLM returns confidence of exactly 0 or 1	Valid per schema	Accepted; passed through
E15	LLM model deprecated by provider	API returns model_not_found	Fail loudly; requires LLM_MODEL env update
E16	LLM response truncated due to max_tokens	Incomplete JSON	Parse fails; retry with increased max_tokens or shorter context
E17	Network timeout to LLM API	Connection error	pg-boss retry with backoff
E18	LLM API key invalid	401 error	Fail loudly; all analysis stops; alert fires

Database Edge Cases

#	Edge Case	Behavior	Handling
E19	Supabase connection pool exhausted	Queries fail	Pool wait timeout; request fails; reconnect

E20	Supabase maintenance window	Brief unavailability	pg-boss retries jobs; webhook returns 500, Slack retries
E21	Message text exceeds TEXT column limit	Extremely rare for Slack	TEXT has no practical limit in PostgreSQL
E22	pgvector index not built	Similarity queries return no results	Graceful degradation: context pack built without rollups
E23	Concurrent INSERT same message	UNIQUE constraint violation	ON CONFLICT DO NOTHING — second insert silently ignored

Application Edge Cases

#	Edge Case	Behavior	Handling
E24	App crashes during backfill	Channel stuck in 'initializing'	On startup: query channels with status='initializing' and re-enqueue backfill
E25	Multiple app instances processing same queue	Job duplication	pg-boss SKIP LOCKED prevents this; only one instance claims each job
E26	Clock skew > 5 minutes	All Slack signatures rejected	Use NTP; monitor signature rejection rate
E27	Memory leak in long-running process	OOM crash	Monitor heap; restart policy in Docker/K8s
E28	Large channel (100k+ messages) in backfill	Slow, memory-heavy	BACKFILL_MAX_PAGES limits scope; stream processing (no in-memory accumulation)

---

24. Worst-Case Scenario Analysis

24.1 Scenario: LLM Provider Complete Outage (4+ hours)

Impact: No new sentiment analysis. Messages continue to be stored. Queue grows.

Timeline:

- T+0: LLM API calls start failing (500/503 errors).
- T+30s: pg-boss retries first failed jobs.
- T+2m: All retry attempts exhausted. Jobs move to dead-letter.
- T+5m: Alert fires: "LLM error rate > 5%".
- T+4h: Queue has accumulated ~2400 unanalyzed messages (at 500 msg/min, only a fraction trigger analysis).

Recovery:

- LLM provider recovers.
- Dead-letter jobs can be replayed manually.
- New messages trigger analysis normally.

- Running summaries may have a gap — next rollup fills it.

Data loss: Zero. Messages are stored regardless of LLM availability.

24.2 Scenario: Supabase Database Unavailable (30 minutes)

Impact: Complete system failure. No messages stored, no events deduplicated, webhook returns 500.

Timeline:

- T+0: Supabase connection fails.
- T+0: Webhook handler cannot INSERT into slack_events or messages.
- T+0: Returns 500 to Slack.
- T+10s: Slack retries the event.
- T+1m: Slack retries again.
- T+5m: Slack gives up on that event (3 retries exhausted).
- T+30m: Database recovers.

Recovery:

- Events that Slack gave up on are lost. This is a ~30 minute gap.
- No way to recover these events from Slack (Events API has no replay mechanism).
- Mitigation: The periodic thread reconciliation (every 5 minutes) will catch up on missed thread replies.
- Mitigation: For missed top-level messages, a manual backfill can be triggered.

Data loss: Messages during the outage window are lost unless manually backfilled.

24.3 Scenario: 50 Channels All Activate Simultaneously

Impact: High CPU, high queue depth, high LLM cost.

Timeline:

- T+0: 50 channels each producing 10 messages/minute = 500 messages/minute total.
- T+0: pg-boss message.ingest worker (concurrency: 8) handles 500 messages/minute easily (each takes <50ms).
- T+0: LLM gating: with 20-message threshold, each channel triggers LLM every ~2 minutes = 25 LLM calls/minute across all channels.
- T+0: LLM analyze worker (concurrency: 4) handles 4 concurrent calls * ~3 seconds each = ~80 calls/minute max. 25 calls/minute is well within capacity.
- T+0: Cost: 25 calls/min * $0.02/call = $0.50/minute = $30/hour.

Bottleneck: LLM API rate limit. OpenAI rate limits depend on tier. Tier 1 allows 500 RPM for GPT-4o-mini. We're at 25 RPM — well within limit.

Mitigation: If more channels activate, increase LLM_MSG_THRESHOLD to 50 or LLM_TIME_THRESHOLD to 30 minutes to reduce call frequency.

24.4 Scenario: Single Channel with 100,000+ Message History

Impact: Slow backfill, high API call count.

Timeline:

- T+0: Bot joins channel. Backfill starts.

- T+0: BACKFILL_MAX_PAGES=100, SLACK_PAGE_SIZE=200 → max 20,000 messages from conversations.history.
- T+5m: History fetch complete (~1 second per API page with rate limit gaps = ~100 seconds minimum, plus thread fetches).
- T+10m: Thread replies fetched for active threads.
- T+12m: User profiles resolved.
- T+15m: Channel marked as 'ready'.

Edge case: If channel has 500 threads with 50+ replies each, thread fetching alone could be 500 API calls * ~1 second = ~8 minutes.

Mitigation: BACKFILL_MAX_PAGES limits total history scope. Thread fetch is parallelized where Slack rate limits allow.

24.5 Scenario: LLM Cost Exceeds Daily Budget

Impact: LLM analysis paused for that workspace.

Trigger: `llm_costs` daily aggregation exceeds `LLM_DAILY_BUDGET_USD` config.

Behavior:

1. Before each `llm.analyze` job, check today's total cost for the workspace.
2. If total >= budget: skip the job, mark as `'skipped'`, log reason.
3. Messages continue to be stored and normalized.
4. Analysis resumes next day when the daily counter resets.
5. Alert fires: "LLM daily budget exceeded for workspace {id}".

---

25. Cost Model and Budget Control

25.1 Per-Analysis Cost Estimates

Model	Input Cost	Output Cost	Typical Call	Estimated Total
GPT-4o-mini	$0.15/1M tokens	$0.60/1M tokens	~800 input, ~200 output	~$0.0003
GPT-4o	$2.50/1M tokens	$10.00/1M tokens	~1500 input, ~300 output	~$0.007
Claude Haiku 4.5	$0.25/1M tokens	$1.25/1M tokens	~800 input, ~200 output	~$0.0005
Claude Sonnet 4.6	$3.00/1M tokens	$15.00/1M tokens	~1500 input, ~300 output	~$0.009
text-embedding-3-small	$0.02/1M tokens	N/A	~200 tokens	~$0.000004

25.2 Monthly Cost Projections

Assuming 50 active channels, 10,000 messages/day, LLM triggered every 20 messages:

```
LLM analysis calls: 10,000 / 20 = 500 calls/day
Cost per call (GPT-4o-mini): $0.0003
Daily analysis cost: $0.15

Rollup calls (channel): 50 channels * 24 rollups/day = 1,200 calls/day
Cost per rollup: $0.001
Daily rollup cost: $1.20

Embedding calls: 1,200 rollups/day * $0.000004 = $0.005/day

Total daily cost: ~$1.35
Monthly cost: ~$40.50
```

This is extremely efficient. Even aggressive usage with GPT-4o for thread analysis adds ~$3.50/day ($105/month).

---

26. Observability and Monitoring

26.1 Structured Logging (pino)

Every log entry includes:

```
{
    "level": "info",
    "time": 1709571234567,
    "correlationId": "uuid",
    "workspaceId": "T12345",
    "channelId": "C12345",
    "action": "backfill:done",
    "duration_ms": 12345,
    "messageCount": 500,
    "threadCount": 23
}
```

Never logged: Raw message text. Only metadata.

26.2 Health Check Endpoint

```
GET /

Response:
{
    "status": "ok",
    "version": "1.0.0",
    "uptime_seconds": 86400,
    "queue": {
        "active": 3,
```

```
        "waiting": 12,
        "failed": 0
    },
    "channels": {
        "total": 50,
        "ready": 48,
        "initializing": 2,
        "failed": 0
    }
}
```

---

27. Deployment Strategy

27.1 Local Development

```
1. Clone repository
2. Copy .env.example to .env, fill in values
3. Create Supabase project, copy connection strings
4. Run migrations: npx node-pg-migrate up
5. Start dev server: pnpm dev
6. Expose via ngrok: ngrok http 3000
7. Configure Slack app with ngrok URL
8. Invite bot to a test channel
```

27.2 Production

```
Docker container (Node.js 20 Alpine)
  → Deployed to Railway / Render / Fly.io / AWS ECS
  → Points to Supabase PostgreSQL (hosted)
  → Environment variables in platform secret manager
  → Health check on GET /
  → Auto-restart on crash
  → Horizontal scaling: single instance for MVP, add instances behind load balancer
for scale
```

27.3 Environment Matrix

Environment	Database	LLM	Purpose
Development	Supabase (dev project)	GPT-4o-mini (dev key)	Local development
Staging	Supabase (staging project)	GPT-4o-mini (staging key)	Pre-production validation
Production	Supabase (prod project)	GPT-4o-mini + GPT-4o	Live system

---

28. Phased Implementation Roadmap

Phase A: Foundation (Weeks 1-2)

Goal: Persistent Supabase storage, modular codebase, pg-boss queue.

- Decompose monolithic index.ts into modules.
- Install dependencies (pg, pg-boss, zod, pino, dotenv, @supabase/supabase-js).
- Create Supabase project and configure.
- Write and run migration 001 (channels, slack_events, messages, thread_edges, user_profiles).
- Replace in-memory Maps with Supabase reads/writes.
- Replace in-memory event dedup with slack_events table.
- Wire pg-boss for channel.backfill and message.ingest jobs.
- Verify: messages persist across restarts.

Phase B: Context Graph + User Profiles (Weeks 3-4)

Goal: Complete thread structure, user names, periodic reconciliation.

- Implement user profile resolution with caching.
- Add bot identity auto-detection via auth.test.
- Implement periodic thread reconciliation (5-minute loop).
- Write migration 002 (message_analytics, channel_state).
- Build channel_state management.
- Add thread and timeline API endpoints.

Phase C: LLM Integration + Sentiment Pipeline (Weeks 5-7)

Goal: Working emotion analysis with conditional gating.

- Install openai, @anthropic-ai/sdk, ajv, node-emoji.
- Build text normalization pipeline.
- Build prompt templates.
- Build emotionService with provider abstraction.
- Build conditional LLM gating.
- Build llm.analyze job handler.
- Write migration 004 (llm_costs).
- Track costs per request.
- Add manual analysis trigger endpoint.
- Build alerting service.

Phase D: Context Management + Summarization (Weeks 8-9)

Goal: Intelligent context packs, running summaries, semantic retrieval.

- Write migration 003 (context_documents with pgvector).
- Build incremental summarizer.
- Build context assembler.
- Build rollup job handlers.
- Implement backfill hierarchical summarization.
- Wire context packs into LLM calls.

Phase E: Analytics + Hardening (Weeks 10-11)

Goal: Production-ready with monitoring, retention, full API.

- Build analytics API endpoints (sentiments, trends, costs).
- Add data retention pg_cron jobs.
- Add input validation (zod) on all API routes.
- Add structured error handling middleware.
- Docker setup.
- Integration tests.
- Load testing.

---

29. Technology Stack Summary

Layer	Technology	Purpose
Runtime	Node.js 20 LTS	Single language backend
Language	TypeScript 5.9 (strict)	Type safety
Web Framework	Express 5.2	HTTP server, webhook handler
Database	Supabase PostgreSQL 15	Managed relational storage
Vector Search	pgvector (via Supabase)	Semantic context retrieval
Scheduled Jobs	pg_cron (via Supabase)	Data retention, cleanup
Job Queue	pg-boss	Async processing, retry, backoff
DB Client	@supabase/supabase-js + pg	Dual: ergonomic + raw SQL
LLM (Primary)	OpenAI API (GPT-4o-mini)	Emotion classification
LLM (Alternative)	Anthropic API (Claude Haiku)	Swappable via env var
LLM SDK	openai (npm)	Official Node.js SDK
Schema Validation	ajv	LLM JSON response validation
Config Validation	zod	Environment variable validation
Emoji Processing	node-emoji	Unicode emoji to text
Logging	pino	Structured JSON logging
Package Manager	pnpm	Fast, disk-efficient
Containerization	Docker (Alpine)	Lightweight deployment

---

30. Appendix: Key Configuration Defaults

```
BACKFILL_DAYS=30                    # How far back to fetch on join
SLACK_PAGE_SIZE=200                 # Messages per API page
BACKFILL_MAX_PAGES=100              # Max pages = 20,000 messages
LLM_PROVIDER=openai                 # openai | anthropic
LLM_MODEL=gpt-4o-mini              # Single-message model
LLM_MODEL_THREAD=gpt-4o            # Thread analysis model
LLM_MSG_THRESHOLD=20               # Messages before auto-trigger
LLM_TIME_THRESHOLD_MIN=10          # Minutes before time-trigger
LLM_COOLDOWN_SEC=60                # Seconds between auto-triggers
LLM_RISK_THRESHOLD=0.7             # Risk score for immediate trigger
LLM_DAILY_BUDGET_USD=10.00         # Max daily LLM spend per workspace
MESSAGE_RETENTION_DAYS=90           # Raw message retention
USER_PROFILE_CACHE_TTL_HOURS=24    # Profile cache duration
THREAD_RECONCILE_INTERVAL_MIN=5    # Thread sync frequency
LOG_LEVEL=info                      # pino log level
```

End of Implementation Plan

Document generated: March 2026 Architecture: LLM-Only | Node.js-Only | Supabase Version: 3.0